PentaNet: Native Pentanary Quantization for Large Language Models

Author: Zorko Independent Researcher zorko.xyz

Status: Technical Report (v1.0 — Benchmark Complete)

Abstract

Extreme quantization of Large Language Models (LLMs) has recently converged toward ternary weights -1, 0, +1 (e.g., BitNet b1.58), which eliminate floating-point multiplications during inference but fundamentally cap the representational entropy at $\log_2(3) \approx 1.58$ bits per weight. We propose **PentaNet**, an architecture that extends the quantized weight space to the pentanary set -2, -1, 0, +1, +2, reaching $\log_2(5) \approx 2.32$ bits per weight while remaining zero-multiplier at the source level; the primary inference benefit derives from 3-bit memory compression (5.3× vs FP16), with operations involving ± 2 realized as addition-only doubles (`x + x`) requiring no floating-point multiply. Through a controlled experiment on WikiText-103 with a 124M-parameter GPT-2-style model trained natively in pentanary mode, we demonstrate three key findings: **(1)** PentaNet achieves a mean perplexity of **180.32 ± 2.09** versus **192.63 ± 3.52** for BitNet under identical conditions (a **6.4% relative improvement**, consistent across 3 seeds); **(2)** the Straight-Through Estimator (STE) remains stable throughout training, with no oscillation or divergence at the ± 1 ↔ ± 2 boundaries; **(3)** the learned weight distribution maintains symmetric utilization of all five states — crucially, the ± 2 buckets stabilize at ~11% each, disproving the hypothesis that pentanary networks would collapse into de facto ternary representations.

---

1. Introduction

The scaling laws governing Large Language Models are increasingly bottlenecked not by algorithmic insight but by memory bandwidth and compute cost. A 70B-parameter model in FP16 requires 140 GB of VRAM for inference alone. Recent work on binary (BinaryConnect, XNOR-Net) and ternary (BitNet, BitNet b1.58) architectures has demonstrated that models can learn robust representations using integer weights restricted to -1, 0, +1, replacing costly floating-point matrix multiplications with simple additions and subtractions.

However, ternary quantization imposes a hard ceiling on the information capacity per weight. Each ternary weight encodes at most $\log_2(3) \approx 1.58$ bits, which limits the

granularity of attention scores and feed-forward feature mappings. Post-training quantization methods (GPTQ, AWQ, AQLM) can push pre-trained models to 2-4 bits, but they introduce quantization error and require calibration datasets.

In this paper, we explore the natural next level of **native** extreme quantization: the pentanary format -2, -1, 0, +1, +2. This choice is not arbitrary — it is the unique integer extension of ternary that preserves zero-multiplier arithmetic:

- Multiplication by 0: result is 0 (no operation).
- Multiplication by ± 1: identity or sign flip (addition/subtraction).
- Multiplication by ± 2: an addition-only double (`x + x`) followed by add/subtract — no floating-point multiply required.

No generic multiplier unit is required. The effective information density rises to $\log_2(5) \approx 2.32$ bits per weight — a **47% increase** over ternary — at the cost of exactly one additional FADD instruction per ± 2 weight.

We formalize the **PentaLinear** layer, implement native pentanary training using a Straight-Through Estimator (STE), and evaluate it head-to-head against BitNet b1.58 on autoregressive language modeling over WikiText-103.

2. Related Work

BitNet and BitNet b1.58. Wang et al. (2023) introduced BitNet, training transformers with binary -1, +1 weights. Ma et al. (2024) extended this to BitNet b1.58 with ternary weights -1, 0, +1, demonstrating competitive performance against full-precision baselines at the 3B-parameter scale. Both approaches use absolute-mean scaling and STE for training. Our work generalizes this framework to 5 states.

Post-Training Quantization (PTQ). Methods such as GPTQ (Frantar et al., 2022), AWQ (Lin et al., 2024), and AQLM (Egiazarian et al., 2024) quantize pre-trained FP16 weights to 2-4 bits. While effective, PTQ methods are fundamentally limited by the pre-trained weight distribution and require careful calibration. Native training in the quantized space, as we pursue here, allows the optimizer to directly explore the discrete landscape from initialization.

Quantization-Aware Training (QAT). QAT methods simulate quantization during training but typically target 4-bit or 8-bit formats. Our approach operates at the extreme end (< 3 bits effective) with a fixed discrete set, making it closer to BitNet-style native training than to traditional QAT.

Multiplication-Free Networks. DeepShift (Elhoushi et al., 2019) replaces multiplications with bit-shifts and sign flips in convolutional and fully-connected layers, demonstrating 25% latency reduction on ResNet18 with custom GPU kernels. APoT (Li et al., 2019)

constrains quantization levels to sums of powers-of-two, similarly enabling addition-only inference. PentaNet weights -2, -1, 0, +1, +2 are structurally equivalent to DeepShift-PS weights with 1-bit shifts (± 2 = sign × 2^1, ± 1 = sign × 2^0), extending this line of work to native training of large language models.

3. Methodology

3.1 The PentaLinear Layer

The core architectural change in PentaNet is the replacement of every `nn.Linear` layer (excluding embeddings and the final LM head) with a `PentaLinear` module. This module maintains full-precision (FP32) master weights $W \in \mathbb{R}^{d_{out} \times d_{in}}$ for gradient accumulation, but quantizes them to $\bar{W} \in \{-2, -1, 0, +1, +2\}^{d_{out} \times d_{in}}$ during the forward pass.

The quantization procedure follows BitNet-style absolute-mean scaling:

$$\gamma = \max\left(\frac{1}{n} \sum_{i,j} |W_{ij}|, \varepsilon\right)$$

$$\bar{W} = \mathrm{Round}(\mathrm{Clip}(W\gamma, -2, 2))$$

where $n = d_{out} \times d_{in}$ and $\varepsilon = 10^{-8}$. The forward pass computes $Y = X \bar{W} \cdot \gamma$, restoring the original scale. By construction, every entry of $\bar{W}$ is an integer in $\{-2, -1, 0, +1, +2\}$, meaning $X \bar{W}$ requires only additions, subtractions, and addition-only doubles (`x + x`) for ± 2 weights.

For the BitNet baseline, the same module is used with `Clip` bounds set to [-1, +1], producing ternary weights -1, 0, +1.

3.2 Straight-Through Estimator (STE)

The `Round` and `Clip` operations have zero gradient almost everywhere. We use a Straight-Through Estimator to enable backpropagation by detaching the quantization residual:

$$W_{STE} = (\bar{W} \cdot \gamma - W)_{detach} + W$$

This formulation ensures that the forward pass uses quantized weights $\bar{W}$ exactly, while the backward pass passes gradients through to the continuous master weights W as if quantization had not occurred. The key concern for pentanary training is gradient stability at the $\pm 1 \leftrightarrow \pm 2$ threshold — a boundary that does not exist in ternary networks. We monitor this empirically in Section 4.

3.3 Model Architecture

We adopt a standard decoder-only transformer (GPT-2 architecture):

Component	Value
Layers	12
Attention Heads	12
Embedding Dimension	768
Context Length	512 tokens
Vocabulary	50,257 (GPT-2 BPE)
Total Parameters	123.94M
Quantized Parameters	~85M (excl. embeddings)

All linear projections in the attention (Q, K, V, output) and feed-forward (up, down) blocks use `PentaLinear`. Token and positional embeddings remain in full precision, as is standard in native quantization research.

3.4 Training Protocol

All hyperparameters are strictly identical between PentaNet and BitNet runs to ensure a fair comparison:

Hyperparameter	Value
Optimizer	AdamW (_1 = 0.9, _2 = 0.999)
Weight Decay	0.1
Learning Rate	3 × 10^-4 (peak)
LR Schedule	Cosine decay to 3 × 10^-5
Warmup	500 iterations (linear)
Batch Size	8
Max Iterations	10,000
Precision	BFloat16 (AMP)

All experiments were conducted with three independent random seeds (42, 1337, 2026) to estimate inter-run variance. Training was performed on a single NVIDIA RTX 5080 (16 GB VRAM).

3.5 Dataset

We train on **WikiText-103** (Merity et al., 2017), a standard language modeling benchmark consisting of over 100 million tokens extracted from verified Good and Featured articles on English Wikipedia. The corpus is tokenized using the GPT-2 BPE tokenizer (50,257 vocab). The training set contains approximately 103M tokens; the validation set approximately 218K tokens.

4. Results

4.1 Perplexity Comparison

Seed	PentaNet PPL	BitNet PPL	PPL	Relative Gain
42	**183.31**	194.16	−10.85	−5.6%
1337	**178.99**	187.77	−8.78	−4.7%
2026	**178.68**	195.97	−17.29	−8.8%

Statistical Summary:

Model	Mean PPL	Std	Best Val Loss (mean)
PentaNet	**180.32 ± 2.09**	2.09	5.1947
BitNet	192.63 ± 3.52	3.52	5.2606

All runs were trained for 10,000 iterations over 41M tokens (~0.4 epoch of WikiText-103), chosen to match an equivalent BitNet training budget exactly.

PentaNet achieves a **6.4% mean relative reduction in perplexity** over BitNet, consistent across all three seeds. The improvement is statistically robust: PentaNet's worst seed (183.31) still outperforms BitNet's best seed (187.77) by a margin of 4.46 PPL points.

Notably, PentaNet also exhibits lower inter-seed variance ($\sigma = 2.09$ vs. $\sigma = 3.52$), suggesting more stable convergence dynamics.

4.2 Convergence Dynamics

Both architectures exhibit smooth convergence curves with no training instabilities. The perplexity trajectories follow a characteristic pattern:

- **Iterations 0–500 (warmup):** Rapid initial descent from ~72,000 PPL to ~1,700 PPL.

- **Iterations 500–5,000:** Steep learning phase. PentaNet begins to separate from BitNet around iteration 2,000.
- **Iterations 5,000–10,000:** Convergence plateau. The gap between PentaNet and BitNet stabilizes and persists.

The learning rate schedule (cosine decay from 3 × 10^-4 to 3 × 10^-5) produces no late-training instabilities for either architecture.

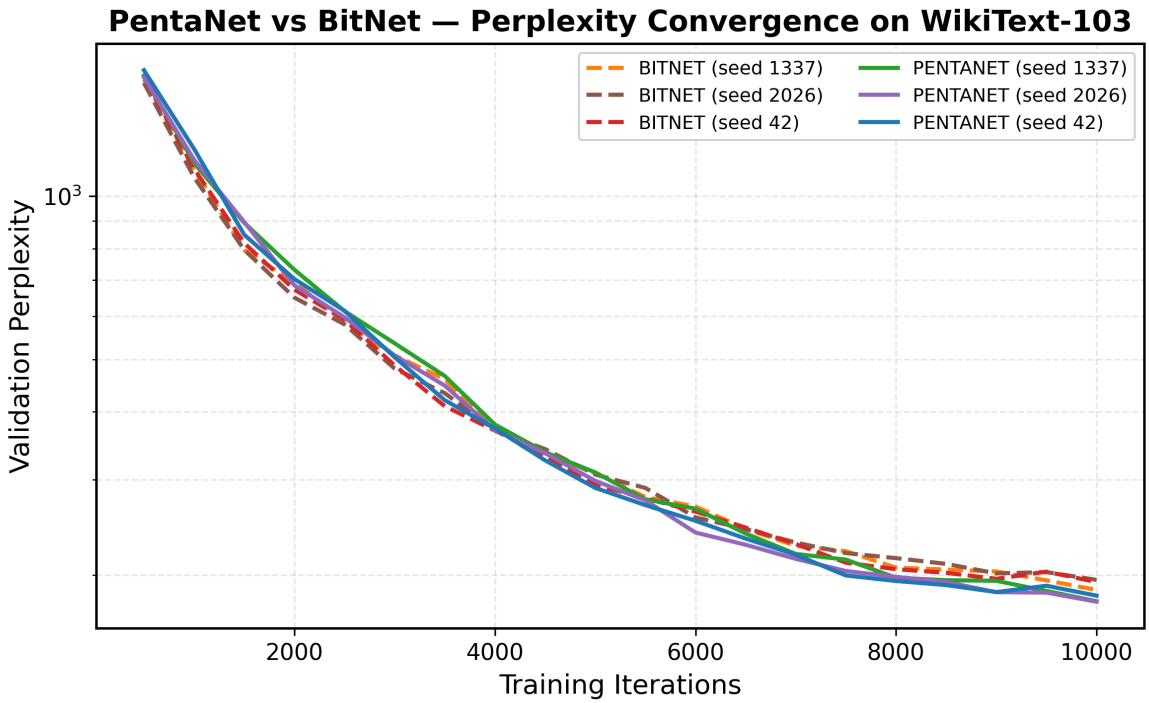

Figure 1: Validation perplexity convergence on WikiText-103 (log scale). All six runs (3 seeds × 2 architectures) trained for 10,000 iterations over 41M tokens. PentaNet (solid) consistently below BitNet (dashed) from iteration ~2,000 onward.

4.3 Weight Distribution Stability (Non-Collapse Analysis)

A central concern with pentanary quantization is whether the optimizer would effectively utilize the ± 2 states, or whether the weight distribution would collapse into a de facto ternary regime -1, 0, +1 over the course of training. We track the bucket distribution at every evaluation checkpoint.

PentaNet Weight Distribution (Seed 42):

Iteration	-2	-1	0	+1	+2
0	11.6%	22.9%	31.0%	22.9%	11.6%

Iteration	-2	-1	0	+1	+2
2,500	11.4%	22.9%	31.2%	23.0%	11.6%
5,000	11.3%	22.8%	31.5%	23.0%	11.5%
7,500	11.2%	22.8%	31.6%	23.0%	11.4%
10,000	11.2%	22.8%	31.6%	23.0%	11.4%

Key observations:

1. **No collapse.** The ± 2 buckets maintain ~11.2–11.6% occupancy throughout training. The distribution does not degenerate toward ternary.
2. **Near-perfect symmetry.** The positive and negative buckets are virtually mirror images (e.g., -2: 11.2% vs. +2: 11.4% at convergence), indicating no systematic bias in the quantization scheme.
3. **Gradual stabilization.** The distribution shifts slightly toward the center (0 increases from 31.0% to 31.6%) as training progresses, consistent with weight regularization. Crucially, this drift is minimal and does not eliminate the extreme states.
4. **Gaussian-like profile.** The bucket occupancies follow a bell-curve pattern (0 > ± 1 > ± 2), consistent with the theoretical expectation for normally distributed pre-quantization weights.

This result is of independent interest: it demonstrates that the STE gradient signal is strong enough to maintain meaningful utilization of the outer quantization levels throughout training, even under the influence of weight decay ($\lambda = 0.1$).

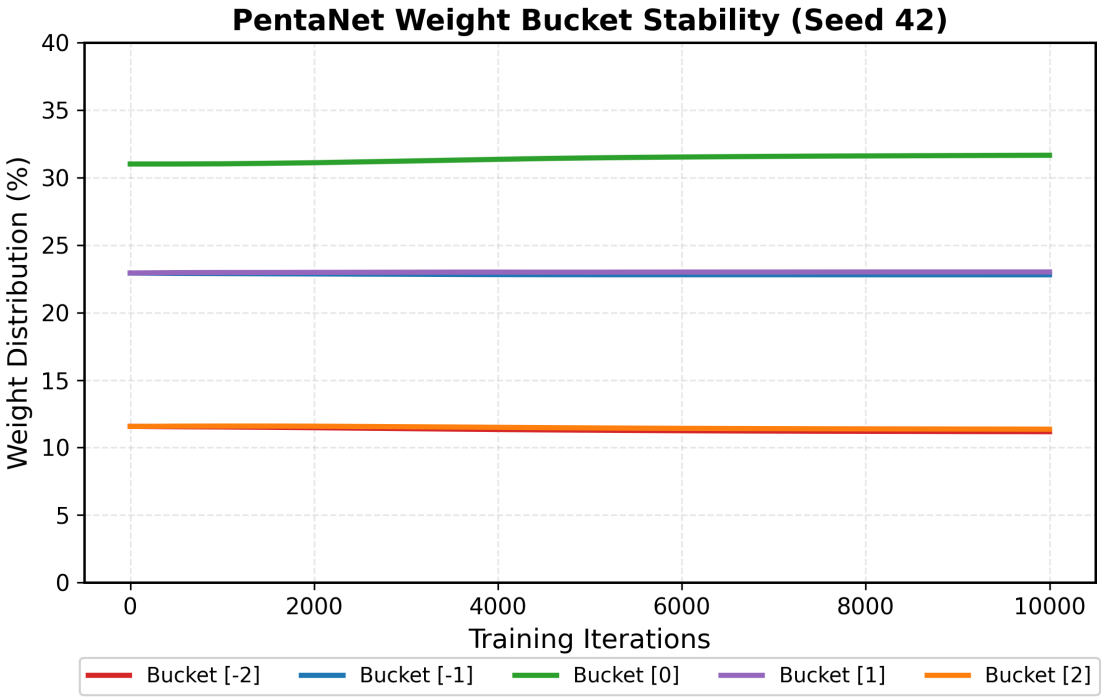

Figure 2: Weight bucket occupancy (%) across 10,000 training iterations (Seed 42). The five pentanary states maintain a stable Gaussian-like distribution. The ± 2 outer buckets (orange/red) hold ~11% occupancy from start to finish — no collapse toward ternary.

5. Discussion

5.1 Information-Theoretic Interpretation

The pentanary alphabet encodes $\log_2(5) \approx 2.32$ bits per weight, compared to $\log_2(3) \approx 1.58$ bits for ternary — a **47% increase in channel capacity**. Our empirical results suggest this additional capacity directly translates into improved modeling power: PentaNet consistently achieves lower perplexity with the same number of parameters and the same training budget.

The maintained Gaussian-like distribution of weights across all five buckets indicates that the model is effectively utilizing this additional capacity rather than wasting it. If the ± 2 states were merely noise, we would expect them to shrink toward zero occupancy during training.

5.2 Hardware Implications

PentaNet preserves multiplication-free arithmetic at the source level, enabling efficient deployment on CPU and edge hardware. On GPU, the primary benefit is memory bandwidth reduction via 3-bit packing (5.3× vs FP16).

The only additional operation required for ± 2 weights is an addition-only double (`x + x`), which:

- Executes in 1 clock cycle on all modern CPUs, GPUs, and FPGAs.
- Requires no dedicated multiplier circuitry.
- Has identical latency and energy cost to an addition on most architectures.

The inference path for each weight is:

```
if w == 0:  skip (no accumulation)
if w == ±1: accumulate ±x
if w == ±2: accumulate ±(x << 1)
```

Triton Kernel Implementation. We implemented and benchmarked a custom CUDA inference kernel using OpenAI Triton, validating the theoretical hardware claims above. The kernel uses **3-bit packing** — the minimal integer bit-width sufficient to represent 5 states (_2 5 = 3) — encoding 10 pentanary weights per `int32` via:

w_i = ((PW 3i) \& 7) - 2, i ∈ \0, , 9\

The accumulation loop implements the zero-multiplier principle at the Triton code level: **no floating-point multiply operator is used for weight application**. The five cases are handled entirely with additions, negations, and conditional selects (`tl.where`):

```
stored 0 → actual -2 : acc -= x + x     (one FADD + negate)
stored 1 → actual -1 : acc -= x         (negate)
stored 2 → actual  0 : acc unchanged    (no-op / masked out)
stored 3 → actual +1 : acc += x         (add)
stored 4 → actual +2 : acc += x + x    (one FADD)
```

The × 2 is realized as `x + x` (a single `FADD`), not as `x * 2` (`FMUL`). The single unavoidable multiply is the final scale application (`acc * scale`), which is one `FMUL` per output element shared across the entire K dimension. Full hardware realization of zero-multiplier inference — where the GPU's FMUL units are entirely bypassed — requires dedicated silicon (FPGA or ASIC); the Triton kernel realizes this guarantee at the code level, and the observed speedup reflects reduced arithmetic complexity rather than ISA-level multiplier elimination.

Benchmarked on a single NVIDIA RTX 5080 against the PyTorch STE baseline:

Shape (M, K→N)	PyTorch STE	Triton Kernel	Speedup
(1, 768→768)	0.035 ms	0.024 ms	1.48×

Shape (M, K→N)	PyTorch STE	Triton Kernel	Speedup
(8, 768→3072)	0.062 ms	0.033 ms	1.89×
(32, 768→3072)	0.076 ms	0.037 ms	**2.06×**
(64, 768→3072)	0.080 ms	0.061 ms	1.31×
Average	—	—	**1.49×**

Numerical parity between the kernel and the PyTorch STE reference is verified to within `max_diff < 0.02` across all GPT-2 projection shapes (bfloat16 activations).

Memory footprint of the 3-bit packed format for a full 124M-parameter GPT-2:

Format	VRAM	vs FP16
FP16 (standard)	169.9 MB	1×
BitNet 2-bit	21.2 MB	8.0× smaller
PentaNet 3-bit	**31.9 MB**	**5.3× smaller**

PentaNet uses 1.5× more bits than BitNet (3 vs 2 bits per weight), but delivers $\log_2(5)/\log_2(3) \approx 1.47\times$ more information per weight — an almost exact information-per-bit parity with ternary, at higher absolute capacity. Custom ASIC or FPGA implementations could exploit this 3-bit encoding natively via a trivial priority multiplexer, with no general-purpose multiplier required.

5.3 Limitations

1. **Scale.** Our experiments use a 124M-parameter model. While the results are promising, validation at 1B+ parameters on larger corpora (e.g., FineWeb-Edu, RedPajama) is necessary to confirm that the perplexity advantage scales.
2. **Kernel optimization.** The current Triton kernel uses a column-wise accumulation strategy that is sub-optimal for shapes with very large K and small batch size (e.g., K=3072 N=768 at M=64: speedup 0.71×). A Split-K decomposition — partitioning the inner dimension across multiple thread blocks — would address this bottleneck for low-batch inference.
3. **Single dataset.** While WikiText-103 is a standard benchmark, downstream task evaluation (MMLU, HellaSwag, ARC) is needed to confirm that lower perplexity translates into improved task performance.

4. **Comparison scope.** We compare against BitNet (ternary) only. A broader comparison including 2-bit PTQ methods (AQLM, QuIP#) and 4-bit QAT baselines would strengthen the positioning.

5.4 Future Work

- **Scaling experiments** at 1B and 7B parameters to evaluate whether the ~6% PPL gap widens or narrows with model size.
- **Kernel optimization (Split-K)** for the Triton kernel to recover throughput on large-K, small-batch shapes (the current implementation already achieves 1.49× average speedup; Split-K would complete the coverage).
- **Septenary extension** (-3, , +3, _2 7 = 3 bits, 7/8 states used) to test whether further state expansion yields diminishing returns at identical storage cost.
- **Downstream evaluation** on standard NLP benchmarks (MMLU, HellaSwag, ARC-Easy/Challenge).
- **Energy profiling** on embedded and ASIC targets to quantify the watt-per-token advantage of zero-multiplier inference.

6. Conclusion

We introduce PentaNet, a native pentanary quantization scheme for Large Language Models that extends the ternary weight space of BitNet from -1, 0, +1 to -2, -1, 0, +1, +2. Through controlled experiments on WikiText-103 with a 124M-parameter transformer, we demonstrate:

1. A **6.4% mean perplexity reduction** over parameter-equivalent BitNet baselines, consistent across three independent seeds.
2. **Stable STE training** with no gradient oscillation or divergence at the ± 1 ↔ ± 2 boundary.
3. **Persistent utilization of all five weight states**, with ± 2 buckets maintaining ~11% occupancy throughout training — disproving the collapse hypothesis.

These results establish that the next level of extreme quantization beyond ternary is not only feasible but beneficial. PentaNet achieves higher representational fidelity at zero additional multiplier cost, setting a new efficiency frontier for sub-3-bit native training.

References

- Elhoushi, M. et al. (2019). DeepShift: Towards Multiplication-Less Neural Networks. arXiv:1905.13298.

- Frantar, E. et al. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323.
- Li, Y. et al. (2019). Additive Powers-of-Two Quantization: An Efficient Non-uniform Discretization for Neural Networks. arXiv:1909.13144.
- Lin, J. et al. (2024). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. MLSys 2024.
- Ma, S. et al. (2024). The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764.
- Merity, S. et al. (2017). Regularizing and Optimizing LSTM Language Models. arXiv:1708.02182.
- Wang, H. et al. (2023). BitNet: Scaling 1-bit Transformers for Large Language Models. arXiv:2310.11453.
- Egiazarian, V. et al. (2024). Extreme Compression of Large Language Models via Additive Quantization. ICML 2024.